

Énoncé du TP :

Dans le cadre de ce travail pratique, on vous demande de concevoir et de réaliser un système embarqué basé sur un microcontrôleur de type PIC16F permettant la mesure et l'affichage de la température ambiante.

Le système utilise un capteur analogique de température (tel que le LM35) connecté à l'entrée analogique AN0 du microcontrôleur.

La valeur analogique issue du capteur doit être convertie en une valeur numérique grâce au module ADC (Analog to Digital Converter) interne du microcontrôleur. Ensuite, cette valeur doit être traitée pour obtenir la température en degrés Celsius. Le résultat final sera affiché sur un écran LCD 16x2 en utilisant le mode 4 bits.

Le programme doit assurer une mise à jour périodique de la température affichée avec un délai d'une seconde. L'ensemble du système doit être programmé en langage C sous l'environnement MPLAB avec le compilateur XC8.

```
1 #include <xc.h> // Bibliothèque principale pour les registres du PIC
2 #include <stdio.h> // Bibliothèque pour les fonctions de formatage (sprintf)
3
4 // Configuration bits (à adapter si besoin)
5 #pragma config FOSC = HS // Oscillateur externe haute vitesse (quartz)
6 #pragma config WDT = OFF // Désactivation du Watchdog Timer
7 #pragma config LVP = OFF // Désactivation de la programmation basse tension
8
9 #define _XTAL_FREQ 8000000 // Définition de la fréquence du PIC (8 MHz) pour les délais
10
11 // LCD pins
12 #define RS LATDbits.LATD0 // Définition de la broche RS connectée à RD0
13 #define EN LATDbits.LATD1 // Définition de la broche EN connectée à RD1
14
15 void LCD_Command(unsigned char cmd); // Prototype fonction envoi commande LCD
16 void LCD_Char(unsigned char data); // Prototype fonction envoi caractère LCD
17 void LCD_Init(); // Prototype fonction initialisation LCD
18 void LCD_String(char *str); // Prototype fonction affichage chaîne
19
20 // ADC init
21 void ADC_Init() {
22     ADCON0 = 0x01; // Active l'ADC et sélectionne le canal AN0
23     ADCON1 = 0x0E; // Configure AN0 comme entrée analogique, les autres en numérique
24     ADCON2 = 0xA9; // Configure justification à droite + temps acquisition + fréquence ADC
25 }
26
27 // Lire ADC
28 int ADC_Read() {
29     ADCON0bits.GO = 1; // Lance la conversion analogique-numérique
30     while(ADCON0bits.GO); // Attend la fin de la conversion
31     return ((ADRESH<<8) + ADRESL); // Combine les registres haut et bas pour obtenir le résultat
32 }
33
34 // LCD functions
35 void LCD_Command(unsigned char cmd) {
36     RS = 0; // Mode commande (RS = 0)
37     LATD = (LATD & 0x0F) | (cmd & 0xF0); // Envoie les 4 bits de poids fort
38     EN = 1; __delay_ms(2); EN = 0; // Impulsion pour valider l'envoi
39
40     LATD = (LATD & 0x0F) | (cmd << 4); // Envoie les 4 bits de poids faible
41     EN = 1; __delay_ms(2); EN = 0; // Impulsion de validation
42 }
43
44 void LCD_Char(unsigned char data) {
45     RS = 1; // Mode donnée (RS = 1)
46     LATD = (LATD & 0x0F) | (data & 0xF0); // Envoie les bits de poids fort
47     EN = 1; __delay_ms(2); EN = 0; // Validation
48
49     LATD = (LATD & 0x0F) | (data << 4); // Envoie les bits de poids faible
50     EN = 1; __delay_ms(2); EN = 0; // Validation
51 }
52
53 void LCD_Init() {
54     TRISD = 0x00; // Configure PORTD en sortie (pour LCD)
55     __delay_ms(20); // Délai de stabilisation du LCD
56     LCD_Command(0x02); // Initialisation en mode 4 bits
57     LCD_Command(0x28); // Configuration LCD : 2 lignes, 5x7
58     LCD_Command(0x0C); // Affichage ON, curseur OFF
59     LCD_Command(0x06); // Déplacement automatique du curseur
```

```

62
63 void LCD_String(char *str) {
64     while(*str) { // Tant que la chaîne n'est pas terminée
65         LCD_Char(*str++); // Envoie chaque caractère puis passe au suivant
66     }
67 }
68
69 // MAIN
70 void main() {
71     char buffer[16]; // Tableau pour stocker le texte à afficher
72     int adc_value; // Variable pour stocker la valeur brute ADC
73     float temperature; // Variable pour stocker la température calculée
74
75     TRISA = 0xFF; // Configure PORTA en entrée (AN0 pour capteur)
76     TRISD = 0x00; // Configure PORTD en sortie (LCD)
77
78     ADC_Init(); // Initialisation du module ADC
79     LCD_Init(); // Initialisation du LCD
80
81     while(1) { // Boucle infinie
82         adc_value = ADC_Read(); // Lecture de la valeur analogique
83
84         // Conversion (ex: LM35 → 10mV/°C)
85         temperature = (adc_value * 5.0 * 100) / 1023; // Conversion en °C
86
87         sprintf(buffer, "Temp: %.2f C", temperature); // Conversion en texte formaté
88
89         LCD_Command(0x80); // Positionne le curseur au début de la 1ère ligne
90         LCD_String(buffer); // Affiche la température sur le LCD
91
92         __delay_ms(1000); // Attente de 1 seconde avant nouvelle lecture
93     }
94 }

```



Output

Trace/Profiling	Project Loading Warning	oic_avec_afficheur (Build, Load)
Data space	used	121h (289) of 800h bytes (14.1%)
Configuration bits	used	4h (4) of 7h words (57.1%)
EEPROM space	used	0h (0) of 100h bytes (0.0%)
ID Location space	used	0h (0) of 8h bytes (0.0%)

make[2]: Leaving directory 'D:/ranii/malek labiadh/oic avec afficheur .X'

BUILD SUCCESSFUL (total time: 3s)

Loading symbols from D:/ranii/malek labiadh/oic avec afficheur .X/dist/default/debug/oic_avec_afficheur_ .X.debug.elf...

Loading code from D:/ranii/malek labiadh/oic avec afficheur .X/dist/default/debug/oic_avec_afficheur_ .X.debug.elf...

Program loaded with pack, PIC18Fxxx_DFP, 1.7.171, Microchip

Loading completed